
IsoCortex

Software Requirement Specification (SRS)

Complete Platform — Monetization Edition

Version 5.1

Lead Architect & Developer:

Shaheer Qureshi

Document Type: Software Requirement Specification
Version: 5.1
Status: Complete — Major Revision
Date: May 17, 2026
Classification: Confidential

Document Control

Revision History			
Ver	Date	Author	Changes
1.0	—	S. Qureshi	Initial draft
2.0	—	S. Qureshi	Expanded API, added ingestion engine
3.0	—	S. Qureshi	Added HNSW, web UI, monetization
4.0	—	S. Qureshi	Comprehensive overhaul, Docker, CI/CD
5.1	May 17, 2026	S. Qureshi	Fixed 12 warnings (NFR renumbering, ambiguities, missing FR sections); added Section 15: Monetization and Commercial Deployment (pricing tiers, multi-tenancy, billing, compliance)

Distribution List		
Role	Name	Access
Lead Architect	Shaheer Qureshi	Full
Stakeholders	—	Full
Development Team	—	Full
QA Team	—	Full

Contents

1 Introduction

1.1 Purpose

This Software Requirement Specification (SRS) defines the complete set of requirements for **IsoCortex v5.0**, a production-grade semantic search platform. IsoCortex transforms heterogeneous files into vector embeddings, builds high-performance HNSW indices, and provides REST, CLI, and web interfaces for semantic querying.

This document serves as the authoritative reference for all stakeholders — developers, QA engineers, project managers, and business analysts — to understand the system’s functional scope, architectural decisions, non-functional constraints, and operational specifications.

Version 5.0 is a comprehensive rewrite that addresses critical production concerns: concurrency safety, memory management, asynchronous operations, user administration, index format stability, and robust error handling.

1.2 Scope

IsoCortex encompasses:

- A **Python-based ingestion engine** supporting 7 document categories (PDFs, DOCX, Markdown, source code, CSV/JSON data, images via OCR, plain text)
- An **HNSW (Hierarchical Navigable Small World)** index for approximate nearest neighbor (ANN) search with $O(\log N)$ query latency
- A **REST API** built on FastAPI with full CRUD for indexes, documents, search, and administration
- A **Web UI** built as a pure Single Page Application (SPA) with Next.js
- A **CLI tool** for scripting and batch operations
- **Authentication and user management** with role-based access control
- **Analytics and monitoring** via SQLite-backed telemetry
- **Docker deployment** with optimized multi-stage builds
- **Export/Import** for index portability

1.3 Definitions, Acronyms, and Abbreviations

Term / Acronym	Definition
ANN	Approximate Nearest Neighbor — algorithms that find approximate results significantly faster than exact search
API	Application Programming Interface
CLI	Command Line Interface
Cosine Similarity	A metric measuring the cosine of the angle between two vectors; range $[-1, 1]$
DOCX	Microsoft Word Open XML Format
Docker	Containerization platform for reproducible deployments
Embedding	A fixed-length numerical vector representing semantic content
FastAPI	Modern async Python web framework
HNSW	Hierarchical Navigable Small World — a graph-based ANN algorithm

Term / Acronym	Definition
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
LRU	Least Recently Used — a cache eviction strategy
MiniLM	A compact sentence transformer model (all-MiniLM-L6-v2)
OCR	Optical Character Recognition
PDF	Portable Document Format
SSE	Server-Sent Events — one-way real-time server push
SPA	Single Page Application — a web app that loads a single HTML page and dynamically updates content via JavaScript
SRS	Software Requirement Specification
SQLite	Embedded relational database engine
UUID	Universally Unique Identifier
WAL	Write-Ahead Logging — a journaling mode for SQLite

1.4 References

- Malkov, Yu A., & Yashunin, D. A. (2018). *Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs*. IEEE Transactions on Pattern Analysis and Machine Intelligence.
- Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence embeddings using Siamese BERT-networks*. EMNLP 2019.
- FastAPI Documentation: <https://fastapi.tiangolo.com>
- Uvicorn ASGI Server: <https://www.uvicorn.org>
- SQLite WAL Mode: <https://www.sqlite.org/wal.html>

1.5 Overview of Document

This SRS is organized as follows:

- **Section 2** — Overall description and system context
- **Section 3** — System architecture and component design
- **Section 4** — Concurrency model and thread safety guarantees
- **Section 5** — Functional requirements (ingestion, indexing, search, API, CLI, Web UI)
- **Section 6** — Authentication and user management
- **Section 7** — Index format versioning and migration
- **Section 8** — Async operations and long-running tasks
- **Section 9** — Non-functional requirements (performance, security, memory)
- **Section 10** — Error handling specification
- **Section 11** — Rate limiting
- **Section 12** — Known limitations
- **Section 13** — Development phases
- **Section 14** — Deployment
- **Section 15** — Monetization and commercial deployment
- **Appendices** — Format specification, configuration reference, error codes, glossary

2 Overall Description

2.1 Product Perspective

IsoCortex is a self-contained, deploy-anywhere semantic search platform. It does not depend on external vector databases (Pinecone, Weaviate, Milvus) or cloud services. All computation happens locally — embedding generation, index construction, and query execution.

The system is designed for:

- **Enterprises** needing private, on-premise semantic search over internal documents
- **Developers** building AI-powered applications requiring local vector search
- **Researchers** performing semantic analysis over large document corpora
- **Small teams** needing a turnkey solution without infrastructure overhead

2.2 Product Functions

1. **Document Ingestion:** Parse 7+ file formats into chunked text with metadata
2. **Embedding Generation:** Convert text chunks to 384-dimensional vectors using MiniLM
3. **Index Construction:** Build HNSW indices with configurable parameters (M , $ef_construction$, ef_search)
4. **Semantic Search:** Query with natural language, receive ranked results with similarity scores
5. **Batch Search:** Execute multiple queries in a single API call
6. **Multi-Index Management:** Create, update, delete, and switch between independent indices
7. **Incremental Updates:** Add and remove documents without full rebuild
8. **User Management:** Administer users with role-based access control
9. **Analytics:** Track search patterns, popular queries, and system health
10. **Export/Import:** Package and transfer complete indices for backup or sharing

2.3 User Characteristics

User Type	Expertise	Interaction Pattern
End User	Basic computer literacy	Web UI for searching documents
Developer	Python, REST APIs	CLI and API for programmatic access
Admin	System administration	Web UI and API for user/system management
Integrator	DevOps, Docker	Docker Compose for deployment

2.4 Constraints

- **Python 3.10+** required for runtime
- **CPU-only** inference (no GPU dependency in v1.0)
- **Single-node** architecture (no distributed search)
- **SQLite** for metadata and analytics (not PostgreSQL — keeps deployment zero-config)

- **No cloud dependencies** for core functionality
- **Maximum practical index size:** ~10M vectors (memory limited)

2.5 Assumptions and Dependencies

- Host machine has at least 4 GB RAM for comfortable operation with small-to-medium indices
- Internet connection available for first-time model download
- File system supports extended attributes (for metadata storage)
- Modern browser (Chrome, Firefox, Safari, Edge) for Web UI
- Docker 20.10+ and Docker Compose v2+ for containerized deployment

3 System Architecture

3.1 High-Level Architecture

IsoCortex follows a layered architecture with clear separation of concerns:

Layer Stack

1. **Presentation Layer** — Web UI (Next.js SPA), CLI (Typer)
2. **API Layer** — FastAPI REST endpoints, authentication middleware
3. **Service Layer** — Search engine, index manager, user manager, job scheduler
4. **Core Layer** — HNSW implementation, embedding provider, extractor pipeline
5. **Storage Layer** — File system (vectors, indices, models), SQLite (metadata, analytics, rate limits)

3.2 Component Diagram

The system comprises the following primary components:

- **Ingestion Engine:** Routes files to category-specific extractors, chunks text, extracts metadata
- **Embedding Provider:** Generates vector embeddings with pluggable model support
- **HNSW Index:** Graph-based ANN index with configurable construction and search parameters
- **Index Manager:** Orchestrates multi-index CRUD, versioning, compaction, and serialization
- **Search Engine:** Handles single and batch semantic search with pagination and filtering
- **API Server:** FastAPI application serving REST endpoints with middleware pipeline
- **Auth Module:** JWT-based authentication with bcrypt password hashing
- **User Manager:** Role-based user administration (admin/user roles)
- **Job Scheduler:** Manages asynchronous long-running tasks (index creation, export/import)
- **Analytics Engine:** SQLite-backed query logging, usage metrics, and health monitoring
- **Rate Limiter:** SQLite-backed sliding window rate limiting per API key and endpoint
- **CLI Interface:** Typer-based command-line tool for all operations
- **Web UI:** Next.js SPA providing visual management and search interface

3.3 Directory Structure

Project Layout

```
isocortex/
  src/
    api/                # FastAPI application
      routes/           # Route handlers
      middleware/       # Auth, rate limiting, logging
      schemas/         # Pydantic models
    core/
      hnsf/             # HNSW implementation
      embedding/        # Embedding provider ABC + MiniLM
      extractor/        # FileExtractor ABC + registry
      search/           # Search engine + batch search
    engine/
      ingestion/        # Ingestion pipeline
      indexing/         # Index manager
      jobs/             # Async job scheduler
      analytics/        # Analytics engine
    auth/               # Authentication + user management
    storage/            # SQLite, file I/O helpers
    config/             # Configuration management
    models/             # Downloaded embedding models
    indices/            # HNSW index data directory
    web/                # Next.js SPA source
    cli/                # Typer CLI entry point
  Dockerfile
  docker-compose.yml
  pyproject.toml
```

3.4 Data Flow

1. User uploads files or provides a directory path
2. Ingestion engine routes files to appropriate extractors based on file extension
3. Extracted text is chunked into semantically meaningful segments (overlap configurable)
4. Chunks are queued for embedding generation
5. Embedding provider converts text chunks to 384-dimensional float32 vectors
6. Vectors and metadata are inserted into the HNSW index
7. Index is serialized to disk (vectors.bin + metadata.json + hnsf_index.bin)
8. Query text is embedded and compared against the index
9. Top-*k* results are returned with similarity scores (normalized per metric), paginated

3.5 Embedding Model Management

Embedding Model Lifecycle

The system manages embedding models with a well-defined lifecycle:

Download on First Run:

- On first startup, IsoCortex downloads the default model (all-MiniLM-L6-v2) from Hugging Face
- Progress indicator shows download percentage and estimated time remaining
- Model is cached in `~/isocortex/models/` for subsequent runs
- SHA-256 checksum validation ensures download integrity

Pluggable Embedding Interface:

- Abstract base class: `EmbeddingProvider` defines `embed(text) -> np.ndarray`

- Default implementation: `MiniLMProvider` using sentence-transformers
- Future: Enterprise tier supports custom embedding models via provider registration

Model Caching:

- Models stored persistently in `~/.isocortex/models/`
- Version-pinned model identifiers prevent silent model changes
- Cache directory structure: `models/{model_id}/config.json`, `model_weights.bin`

EmbeddingProvider ABC

```
from abc import ABC, abstractmethod
import numpy as np

class EmbeddingProvider(ABC):
    @abstractmethod
    def embed(self, text: str) -> np.ndarray:
        """Embed a single text string into a vector."""
        pass

    @abstractmethod
    def embed_batch(self, texts: list[str]) -> np.ndarray:
        """Embed multiple text strings into vectors."""
        pass

    @abstractmethod
    def dimension(self) -> int:
        """Return the embedding dimension."""
        pass

    @abstractmethod
    def model_id(self) -> str:
        """Return the unique model identifier."""
        pass
```

3.6 Plugin Architecture for Extractors

Extractor Plugin System

IsoCortex uses a registry-based plugin architecture for document extractors, enabling extensibility and community contributions.

FileExtractor ABC:

- `supported_extensions() -> list[str]`: Declares which file extensions this extractor handles
- `extract(file_path: Path) -> list[ExtractedChunk]`: Parses a file into text chunks with metadata
- `priority() -> int`: Determines load order when multiple extractors support the same extension (lower = higher priority)

Registry Pattern:

- `ExtractorRegistry` maintains a global registry of all extractors
- `register(extractor: FileExtractor)`: Registers an extractor class
- `get_extractor(extension: str) -> FileExtractor`: Returns the highest-priority extractor for a given extension
- Extractors are auto-discovered from built-in modules at startup

Built-in Extractors:

- `PdfExtractor`: PDF parsing via PyMuPDF

- `DocxExtractor` : DOCX parsing via `python-docx`
- `MarkdownExtractor` : Markdown parsing with front-matter support
- `CodeExtractor` : Source code with AST-aware chunking
- `DataExtractor` : CSV/JSON tabular data
- `ImageExtractor` : OCR via Tesseract for scanned documents
- `TextExtractor` : Plain text with encoding detection

Custom Extractors:

- User-provided extractors loaded from `~/.isocortex/extractors/`
- Each custom extractor is a Python module implementing `FileExtractor`
- Loaded at startup with automatic validation

FileExtractor ABC and Registry

```
from abc import ABC, abstractmethod
from pathlib import Path
from dataclasses import dataclass

@dataclass
class ExtractedChunk:
    text: str
    metadata: dict
    page: int | None = None
    chunk_index: int = 0

class FileExtractor(ABC):
    @abstractmethod
    def supported_extensions(self) -> list[str]:
        pass

    @abstractmethod
    def extract(self, file_path: Path) -> list[ExtractedChunk]:
        pass

    def priority(self) -> int:
        return 100 # default priority

class ExtractorRegistry:
    _extractors: dict[str, FileExtractor] = {}

    @classmethod
    def register(cls, extractor: FileExtractor) -> None:
        for ext in extractor.supported_extensions():
            if ext not in cls._extractors or \
                extractor.priority() < cls._extractors[ext].priority():
                cls._extractors[ext] = extractor

    @classmethod
    def get_extractor(cls, extension: str) -> FileExtractor | None:
        return cls._extractors.get(extension.lower())
```

4 Concurrency Model

4.1 Overview

IsoCortex is designed to handle concurrent access safely across all modules. The concurrency model addresses three primary concerns: **HNSW index thread safety**, **embedding request handling**, and **database concurrent access**. The API server itself runs under Uvicorn with configurable worker processes.

4.2 HNSW Index: ReadWriteLock

The HNSW index uses a **ReadWriteLock** (reader-writer lock) pattern to allow multiple concurrent readers while ensuring exclusive access for writers:

HNSW Concurrency Guarantees

- **Multiple concurrent readers:** Search queries execute simultaneously without blocking each other
- **Exclusive writer:** Only one write operation (insert, delete, update) can modify the index at a time
- **Writer priority:** Pending write operations are given priority over new read locks to prevent writer starvation
- **Read-during-write safety:** Readers that acquired a lock before a writer began continue safely; new readers wait until the write completes

ReadWriteLock Implementation Sketch

```
import threading

class ReadWriteLock:
    def __init__(self):
        self._lock = threading.Lock()
        self._readers = threading.Condition(self._lock)
        self._reader_count = 0
        self._writer_active = False
        self._writer_waiting = 0

    def acquire_read(self):
        with self._readers:
            while self._writer_active or self._writer_waiting > 0:
                self._readers.wait()
            self._reader_count += 1

    def release_read(self):
        with self._readers:
            self._reader_count -= 1
            if self._reader_count == 0:
                self._readers.notify_all()

    def acquire_write(self):
        with self._readers:
            self._writer_waiting += 1
            while self._reader_count > 0 or self._writer_active:
                self._readers.wait()
            self._writer_waiting -= 1
            self._writer_active = True

    def release_write(self):
        with self._readers:
            self._writer_active = False
            self._readers.notify_all()
```

4.3 Embedder Request Queue

The embedding model (MiniLM) is **not thread-safe** by default. IsoCortex uses a single-threaded model inference queue with async request handling:

- **Single inference thread:** All embedding computations run on a dedicated thread to avoid race conditions
- **Async queue:** API handlers submit embedding requests to an `asyncio.Queue` and await results via `asyncio.Future`

- **Batch optimization:** Multiple pending requests are batched together for efficient GPU/CPU inference (even on CPU, batched inference is faster than sequential)
- **Backpressure:** Queue has a maximum depth; when full, new requests receive a 503 Service Unavailable response
- **Embedding cache:** LRU cache (configurable, default 1000 entries) maps `str(query)` to `np.ndarray`, eliminating repeated computations

Embedding Queue Sketch

```
import asyncio
from collections import OrderedDict
import numpy as np

class EmbeddingQueue:
    def __init__(self, provider: EmbeddingProvider, cache_size: int = 1000):
        self._provider = provider
        self._queue: asyncio.Queue = asyncio.Queue(maxsize=500)
        self._cache: OrderedDict[str, np.ndarray] = OrderedDict()
        self._cache_size = cache_size
        self._running = False

    async def embed(self, text: str) -> np.ndarray:
        # Check cache first
        key = text
        if key in self._cache:
            self._cache.move_to_end(key)
            return self._cache[key]

        # Submit to queue and wait
        future = asyncio.get_event_loop().create_future()
        await self._queue.put((key, future))
        result = await future

        # Update cache
        self._cache[key] = result
        if len(self._cache) > self._cache_size:
            self._cache.popitem(last=False)
        return result

    async def _worker(self):
        while self._running:
            batch = []
            futures = []
            # Collect up to 32 pending requests
            try:
                item = self._queue.get_nowait()
                batch.append(item[0])
                futures.append(item[1])
                while len(batch) < 32:
                    item = self._queue.get_nowait()
                    batch.append(item[0])
                    futures.append(item[1])
            except asyncio.QueueEmpty:
                pass

            if batch:
                vectors = self._provider.embed_batch(batch)
                for f, v in zip(futures, vectors):
                    f.set_result(v)
            else:
                await asyncio.sleep(0.01)
```

4.4 SQLite WAL Mode

The analytics database uses SQLite in **WAL (Write-Ahead Logging)** mode for concurrent read/write access:

- **Concurrent reads:** Multiple readers can query the database simultaneously without block-

ing

- **Single concurrent writer:** One writer can write while readers continue reading
- **No reader-writer blocking:** Readers do not block writers and writers do not block readers
- **Durability:** WAL mode provides crash recovery without sacrificing performance
- **Connection pool:** Uses `sqlite3.connect()` with `check_same_thread=False` and a connection pool of 5 connections

Configuration:

```
import sqlite3

conn = sqlite3.connect("analytics.db")
conn.execute("PRAGMA journal_mode=WAL")
conn.execute("PRAGMA busy_timeout=5000")
conn.execute("PRAGMA synchronous=NORMAL")
conn.execute("PRAGMA wal_autocheckpoint=1000")
```

4.5 API Server Concurrency

The API server uses **Uvicorn** with configurable worker count:

- Default: single worker (`uvicorn isocortex.api:app`)
- Production: multi-worker (`uvicorn isocortex.api:app -workers 4`)
- State shared across workers via SQLite (rate limits, analytics) and file system (indices)
- Each worker maintains its own in-memory embedding cache (no cross-worker cache sharing)
- HNSW ReadWriteLock operates per-process; file-based locking coordinates cross-process access when needed

4.6 Thread Safety Summary

Module	Mechanism	Guarantee
HNSW Index	ReadWriteLock	Multi-reader, exclusive writer; no data corruption
Embedding Provider	Single worker thread + queue	Sequential inference; thread-safe by design
Embedding Cache	OrderedDict (in-process)	Safe within single process; no cross-process sharing
SQLite Analytics	WAL mode + pool	Concurrent reads, serialized writes
SQLite Rate Limits	WAL mode + pool	Cross-worker visibility; persistent across restarts
Index Manager	ReadWriteLock + file locks	Safe cross-process index access
Job Scheduler	asyncio event loop	Single event loop per worker; job state in SQLite

5 Functional Requirements

5.1 Document Ingestion

5.1.1 FR-ING-001: File Category Support

The ingestion engine supports 7 document categories, each handled by a dedicated extractor plugin:

Category	Extensions	Extractor	Processing Notes
PDF	.pdf	PdfExtractor	Page-by-page extraction; OCR fallback for scanned pages
Word	.docx	DocxExtractor	Paragraph-level extraction; preserves headers/styles
Markdown	.md, .rst	MarkdownExtractor	Front-matter parsing; code block preservation
Source Code	.py, .js, .ts, .java, .go, .rs, .cpp, .c, .h	CodeExtractor	AST-aware chunking; syntax-aware splitting
Data	.csv, .json, .tsv	DataExtractor	Row-level or record-level chunking
Image	.png, .jpg, .jpeg, .tiff, .bmp	ImageExtractor	OCR via Tesseract; configurable language
Plain Text	.txt, .log, .cfg, .ini	TextExtractor	Encoding auto-detection; line/paragraph chunking

5.1.2 FR-ING-002: Text Chunking

Documents are split into chunks optimized for embedding:

- **Default chunk size:** 512 tokens (approximately 350–400 words)
- **Overlap:** 50 tokens between consecutive chunks to preserve context at boundaries
- **Configurable:** Both chunk size and overlap are configurable per index
- **Metadata preservation:** Each chunk retains source file name, page/section, chunk index, and timestamps
- **Minimum chunk size:** Chunks shorter than 10 tokens are merged with adjacent chunks

5.1.3 FR-ING-003: Directory Ingestion

- Recursive directory traversal with configurable depth limit
- Glob-based include/exclude patterns (`**/*.pdf` , `!**/node_modules/**`)
- Maximum file size limit (default: 50 MB per file)
- Skip duplicate files detected by SHA-256 hash
- Progress reporting: files processed / total files found

5.2 Embedding Generation

5.2.1 FR-EMB-001: Default Model

- Model: **all-MiniLM-L6-v2** (Sentence Transformers)
- Dimensions: 384 (float32)
- Model size: ~80 MB

- Inference speed: ~1000 embeddings/second on modern CPU (batched)
- Quality: Good balance of quality and speed for general-purpose semantic search

5.2.2 FR-EMB-002: Embedding Cache

LRU Embedding Cache

To avoid redundant embedding computations, IsoCortex implements an in-memory LRU cache:

- **Key:** `str(query_text)` (exact string match)
- **Value:** `np.ndarray` of shape (384,)
- **Default size:** 1000 entries
- **Configurable:** `embedding.cache_size` in configuration
- **Invalidation:** Cache is cleared entirely when the embedding model changes
- **Thread safety:** `OrderedDict` protected by the single embedder thread (no additional locking needed)
- **NFR target:** >80% cache hit rate for repeated queries within a session

5.3 HNSW Index

5.3.1 FR-IDX-001: Index Construction

HNSW indices are constructed with the following configurable parameters:

Parameter	Default	Range	Description
M	16	4–128	Number of bidirectional links per layer
<i>ef_construction</i>	200	50–2000	Size of dynamic candidate list during construction
<i>ef_search</i>	50	10–500	Size of dynamic candidate list during search
<code>max_elements</code>	100000	100–10M	Maximum number of vectors in the index
<code>metric</code>	cosine	cosine / l2 / ip	Distance metric for similarity computation

5.3.2 FR-IDX-002: Soft Delete Pattern

Soft Delete for HNSW Vectors

Deleting vectors from an HNSW graph without a full rebuild is achieved through a soft-delete (tombstone) pattern:

Mechanism:

- Each vector has a `deleted: bool` flag in the metadata store
- **Tombstone marking:** Delete operations set `deleted = True` rather than removing the vector from the graph
- **Search filtering:** Query execution skips tombstoned vectors during candidate evaluation
- **No structural change:** The HNSW graph topology remains unchanged during soft delete

Compaction:

- Triggered when tombstoned vectors exceed a configurable threshold (default: 10% of total vectors)
- Compaction rebuilds the HNSW graph including only non-tombstoned entries
- Runs as an async job (non-blocking) — returns `job_id` for progress tracking
- Compaction result: reduced memory footprint, restored search precision
- **Threshold:** `compaction.threshold = 0.10` (10% tombstoned triggers compaction)

Advantages:

- ✓ Instant deletion (no rebuild latency)
- ✓ Batch deletions are $O(1)$ per vector (metadata update only)
- ✓ Search performance degrades gracefully (minor overhead for tombstone check)
- ✓ Eliminates the need for full rebuild on every deletion

5.3.3 FR-IDX-003: Incremental Updates

- New vectors can be added to an existing index without reconstruction
- Soft-deleted vectors are filtered at search time
- Compaction optimizes the graph periodically (see FR-IDX-002)
- Index metadata (file mappings, timestamps) updated atomically

5.4 REST API

5.4.1 API Design Principles

- **RESTful:** Resource-oriented with standard HTTP verbs
- **Consistent paths:** All index-scoped endpoints use the path parameter pattern `/api/v1/indexes/{name}/...`
- **JSON:** All request/response bodies use JSON
- **Versioned:** API prefix `/api/v1/` for forward compatibility
- **Pagination:** Search results support `page/page_size` parameters
- **Error format:** Standardized error envelope with `request_id`
- **Async jobs:** Long-running operations return 202 Accepted with `job_id`

5.4.2 FR-API-001: Index Management Endpoints

Verb	Endpoint	Auth	Description
POST	<code>/api/v1/indexes</code>	JWT	Create new index (returns 202 + <code>job_id</code>)
GET	<code>/api/v1/indexes</code>	JWT	List all indexes with metadata
GET	<code>/api/v1/indexes/{name}</code>	JWT	Get index details and statistics
PUT	<code>/api/v1/indexes/{name}</code>	JWT	Update index configuration
DELETE	<code>/api/v1/indexes/{name}</code>	JWT	Delete index (returns 202 + <code>job_id</code>)

Create Index Request:

```
{
```



```

"name": "my-docs",
"description": "Company documentation",
"files": ["/path/to/documents"],
"exclude_patterns": ["**/node_modules/**"],
"embedding": {
  "model": "all-MiniLM-L6-v2",
  "chunk_size": 512,
  "chunk_overlap": 50
},
"hnsw": {
  "M": 16,
  "ef_construction": 200,
  "ef_search": 50,
  "metric": "cosine"
}
}

```

5.4.3 FR-API-002: Search Endpoints

Verb	Endpoint	Auth	Description
POST	/api/v1/indexes/{name}/search	JWT	Semantic search with pagination
POST	/api/v1/indexes/{name}/search/batch	JWT	Batch semantic search (max 50 queries)

Single Search Request Body:

```

{
  "query": "How does authentication work?",
  "k": 10,
  "page": 1,
  "page_size": 10,
  "filters": {
    "file_extension": [".pdf", ".md"],
    "source": "internal-docs"
  }
}

```

Single Search Response:

```

{
  "request_id": "uuid-here",
  "results": [
    {
      "id": "doc-chunk-uuid",
      "text": "Authentication is handled by...",
      "metadata": {
        "source_file": "auth.md",
        "page": 3,
        "chunk_index": 2,
        "file_extension": ".md",
        "metric": "cosine"
      },
      "score": 0.9234,
      "rank": 1
    }
  ],
  "pagination": {
    "total_results": 47,
    "page": 1,
    "page_size": 10,
    "total_pages": 5
  }
}

```

Score Semantics: The `score` field is always normalized so that **higher values indicate greater similarity**, regardless of the index's `metric` setting. For `cosine`, the raw cosine similarity is returned ($[-1, 1]$, higher = more similar). For `l2`, the score is computed as $1/(1 + distance)$, yielding a $(0, 1]$ range where higher = more similar. For `ip` (inner product), the raw dot product is returned (unbounded; higher = more similar). The index's active metric is included in every search response under `metadata.metric`.

5.4.4 FR-API-003: Pagination Specification

Pagination Design

Search results support **offset-based pagination** with an optional **cursor-based** mode for large result sets:

Offset-Based (default):

- `page`: Page number, starting from 1 (default: 1)
- `page_size`: Results per page, 1–100 (default: 10)
- Response includes: `total_results`, `page`, `page_size`, `total_pages`
- Suitable for result sets up to ~100K results

Cursor-Based (optional):

- Activate by setting `"pagination": "cursor"` in the search request
- Returns `next_cursor` and `has_more` in the response
- Subsequent requests include `"cursor": "<next_cursor>"`
- Suitable for streaming through very large result sets without offsets
- No `total_pages` field (cursor pagination cannot determine total count efficiently)

Constraints:

- `page_size` maximum: 100 (server-enforced)
- `k` must be $\geq$ `page_size` to ensure enough candidates
- If `k < page_size`, server auto-adjusts `k` to `page_size` and logs a warning

5.4.5 FR-API-004: Document Endpoints

Verb	Endpoint	Auth	Description
GET	<code>/api/v1/indexes/{name}/documents</code>	JWT	List all documents in index (paginated)
GET	<code>/api/v1/indexes/{name}/documents/{id}</code>	JWT	Get specific document chunk by ID
POST	<code>/api/v1/indexes/{name}/documents</code>	JWT	Add documents to existing index
DELETE	<code>/api/v1/indexes/{name}/documents/{id}</code>	JWT	Soft-delete a document chunk

5.4.6 FR-API-005: Batch Search Specification

Batch Search API

The batch search endpoint enables efficient execution of multiple queries in a single API call:

Endpoint: `POST /api/v1/indexes/{name}/search/batch`

Request:

```
{
  "queries": [
    {"query": "How does auth work?", "k": 5},
    {"query": "API rate limits", "k": 3},
    {"query": "Docker deployment", "k": 10}
  ],
  "timeout": 30
}
```

Constraints:

- `max_queries` : 50 per batch request
- `timeout` : Per-query timeout in seconds (default: 30s, max: 120s)
- Each query can specify its own `k` value and `filters`
- `page` and `page_size` apply per-query within each result

Partial Success:

- Valid queries execute and return results
- Invalid queries return an error object in their response slot (not a 4xx for the whole request)
- Overall HTTP status is 207 Multi-Status if some queries failed
- Overall HTTP status is 200 if all queries succeeded

Processing:

- Queries are processed in parallel with configurable concurrency (default: 4 concurrent)
- Embedding computations are batched across queries for efficiency
- Total batch timeout is `max(timeout * num_queries / concurrency, timeout)`

Response:

```
{
  "request_id": "uuid-here",
  "results": [
    {
      "query": "How does auth work?",
      "status": "success",
      "results": [...],
      "pagination": {...}
    },
    {
      "query": "API rate limits",
      "status": "success",
      "results": [...],
      "pagination": {...}
    },
    {
      "query": "Invalid query with special chars!!!",
      "status": "error",
      "error": {
        "code": 400,
        "message": "Query too short (minimum 3 characters)"
      }
    }
  ]
}
```

5.4.7 FR-API-006: Export/Import Endpoints

Verb	Endpoint	Auth	Description
POST	<code>/api/v1/indexes/{name}/export</code>	JWT	Export index as .isocortex archive (202 + job_id)
POST	<code>/api/v1/indexes/import</code>	JWT	Import .isocortex archive (202 + job_id)
GET	<code>/api/v1/jobs/{job_id}</code>	JWT	Check export/import job status

5.5 CLI Interface

5.5.1 FR-CLI-001: Command Structure

CLI Commands

```
isocortex index create --name my-docs --path ./documents
isocortex index list
isocortex index info my-docs
isocortex index delete my-docs
isocortex index export my-docs --output ./archive.isocortex
isocortex index import ./archive.isocortex

isocortex search my-docs --query "How does auth work?" --top-k 5

isocortex serve --host 0.0.0.0 --port 8000 --workers 4

isocortex user create --username admin --role admin
isocortex user list
isocortex user delete <user_id>
```

5.6 Web UI

5.6.1 FR-WEB-001: Pure SPA Architecture

Web UI Design Decisions

The Web UI is built as a **pure Single Page Application (SPA)** using Next.js with **client-side routing only**:

- **No SSR**: Next.js is used purely as a React framework with client-side routing via `next/router`
- **Static export**: Supports `output: 'export'` in `next.config.js` for air-gapped deployment scenarios
- **API consumption**: All data comes from the REST API; the UI is a pure client
- **Authentication**: JWT tokens stored in `httpOnly` cookies or `localStorage`
- **State management**: React Context or Zustand for client-side state
- **Build output**: Static HTML/CSS/JS served from the API server or a standalone file server

Key Pages:

- **Dashboard**: Overview of indexes, recent searches, system health
- **Search**: Query interface with filters, pagination, and result display
- **Indexes**: Create, browse, and manage indexes
- **Documents**: Browse and inspect individual document chunks

- **Users:** User administration (admin only)
- **Settings:** Configuration panel
- **Analytics:** Search analytics and usage metrics

5.7 Analytics and Monitoring

5.7.1 FR-ANL-001: Analytics Engine

Analytics Specification

IsoCortex provides a SQLite-backed analytics engine that tracks system usage and enables data-driven operational decisions.

Tracked Metrics:

- **Search queries:** Query text (hashed), result count, latency, index name, timestamp, user ID
- **Index operations:** Create, update, delete, compact, export, import — each with duration and status
- **System health:** Uptime, memory usage, active index count, total vector count, embedding cache hit rate
- **User activity:** Login/logout events, failed auth attempts, API usage per user
- **Error rates:** Error counts by code, endpoint, and time window

Analytics API Endpoints:

- `GET /api/v1/analytics/overview` — System-wide summary (total searches, indexes, users, uptime)
- `GET /api/v1/analytics/search?from=...to=...` — Search analytics with date range filtering
- `GET /api/v1/analytics/indexes` — Per-index statistics (vector counts, query counts, sizes)
- `GET /api/v1/analytics/health` — Real-time system health metrics (memory, CPU, cache stats)

Retention:

- Analytics records retained for a configurable period (default: 90 days)
- Aggregated summaries preserved indefinitely (daily rollups)
- Cleanup runs as a periodic background task (every 24 hours)

Privacy:

- Query text is SHA-256 hashed before storage (irreversible, privacy-preserving)
- User IDs are stored as UUIDs (no PII)
- IP addresses are not logged

5.8 Job Management API

5.8.1 FR-API-007: Job and Admin Endpoints

Verb	Endpoint	Auth	Description
GET	<code>/api/v1/jobs/{job_id}</code>	JWT	Get job status, progress, and result

Verb	Endpoint	Auth	Description
GET	/api/v1/jobs/{job_id}/stream	JWT	SSE stream for real-time progress
GET	/api/v1/jobs	JWT	List all jobs with optional status filter
DELETE	/api/v1/jobs/{job_id}	Admin	Cancel a running job
GET	/api/v1/admin/rate-limits	Admin	View all rate limit statuses
POST	/api/v1/admin/rate-limits/{key}	Admin	Reset rate limit for a specific key

6 Authentication and User Management

6.1 Authentication Overview

IsoCortex uses **JWT (JSON Web Token)** based authentication:

- **Login:** POST /api/v1/auth/login with `username` and `password`
- **Token:** Returns `access_token` (JWT, 24-hour expiry) and `refresh_token` (7-day expiry)
- **Refresh:** POST /api/v1/auth/refresh with `refresh_token`
- **Logout:** POST /api/v1/auth/logout invalidates the refresh token
- **Authorization:** `Authorization: Bearer <token>` header on all protected endpoints

6.2 Password Requirements

Password Policy

- **Minimum length:** 12 characters
- **Complexity:** Must include at least one uppercase letter, one lowercase letter, one number, and one special character
- **Hashing:** bcrypt with cost factor 12 (provides ~250ms hash time on modern hardware)
- **Storage:** Only the bcrypt hash is stored; plaintext passwords are never persisted
- **Validation:** Server-side validation on create and password change; client-side hints in UI

6.3 Account Lockout Policy

- **Failed attempt tracking:** Each username has a counter stored in SQLite
- **Lockout threshold:** 5 consecutive failed login attempts
- **Lockout duration:** 15 minutes
- **Counter reset:** Successful login resets the failed attempt counter
- **Response:** Locked accounts receive 403 Forbidden with `"detail": "Account locked. Try again in X minutes"`
- **Admin override:** Admins can manually unlock accounts via the user management API

6.4 First-Run Setup Wizard

On initial startup (no users in the database):

1. System detects zero users in the user table

2. Setup wizard endpoint becomes available: POST `/api/v1/auth/setup`
3. First user created automatically receives the `admin` role
4. Setup wizard is disabled after the first user is created (returns 409 Conflict)
5. Admin is prompted to create a strong password meeting all complexity requirements

6.5 User Management API

User Management Endpoints

All user management endpoints require JWT authentication. Administrative operations require the caller to have the `admin` role.

Verb	Endpoint	Auth	Description
POST	<code>/api/v1/auth/setup</code>	None (first run only)	Create initial admin user
POST	<code>/api/v1/auth/login</code>	None	Authenticate and receive tokens
POST	<code>/api/v1/auth/refresh</code>	refresh_token	Refresh access token
POST	<code>/api/v1/auth/logout</code>	JWT	Invalidate refresh token
POST	<code>/api/v1/auth/users</code>	Admin	Create a new user
GET	<code>/api/v1/auth/users</code>	Admin	List all users
PUT	<code>/api/v1/auth/users/{id}/password</code>	JWT (self or admin)	Change user password
PUT	<code>/api/v1/auth/users/{id}/role</code>	Admin	Change user role
POST	<code>/api/v1/auth/users/{id}/unlock</code>	Admin	Unlock locked account
DELETE	<code>/api/v1/auth/users/{id}</code>	Admin	Delete a user
GET	<code>/api/v1/auth/me</code>	JWT	Get current user profile

Create User Request (Admin):

```
{
  "username": "analyst1",
  "password": "SecurePassw0rd!2025",
  "role": "user",
  "email": "analyst@company.com"
}
```

User Roles:

Role	Permissions
admin	Full access: user management, index CRUD, search, analytics, settings, export/import
user	Search, view indexes and documents, export data, change own password

7 Index Format Versioning

7.1 Overview

Index files on disk include a version identifier to enable forward compatibility and safe migration. As IsoCortex evolves, the binary format of index data may change. The versioning

system ensures that older indices can be loaded, migrated, or explicitly rejected with clear error messages.

7.2 Format Version Field

Every index includes an `index_info.json` file with a `format_version` field:

```
{
  "format_version": 1,
  "name": "my-docs",
  "created_at": "2025-01-15T10:30:00Z",
  "updated_at": "2025-01-15T12:00:00Z",
  "embedding_model": "all-MiniLM-L6-v2",
  "embedding_dimension": 384,
  "vector_count": 15234,
  "hnsw_params": {
    "M": 16,
    "ef_construction": 200,
    "metric": "cosine"
  },
  "chunk_config": {
    "chunk_size": 512,
    "chunk_overlap": 50
  }
}
```

7.3 Version Negotiation

On index load, the system performs version negotiation:

1. Read `format_version` from `index_info.json`
2. Compare against the **current supported version**
3. **Exact match**: Load index normally
4. **Minor version difference**: Auto-migrate (see breaking change policy below)
5. **Major version difference**: Reject with a clear error message

Version Negotiation Logic

```
SUPPORTED_VERSION = 1

def load_index(index_path: Path) -> HNSWIndex:
    info = json.loads((index_path / "index_info.json").read_text())
    version = info["format_version"]

    if version == SUPPORTED_VERSION:
        return HNSWIndex.load(index_path)

    if version > SUPPORTED_VERSION:
        raise IndexVersionError(
            f"Index format v{version} is newer than supported v{SUPPORTED_VERSION}. "
            f"Please update IsoCortex to the latest version."
        )

    # version < SUPPORTED_VERSION
    migration = MigrationRegistry.get_migration(version, SUPPORTED_VERSION)
    if migration and migration.automatic:
        logger.info(f"Auto-migrating index from v{version} to v{SUPPORTED_VERSION}")
        migration.migrate(index_path)
        return HNSWIndex.load(index_path)
    else:
        raise IndexVersionError(
            f"Index format v{version} requires manual migration. "
            f"Run: isocortex index migrate {index_path} --target {SUPPORTED_VERSION}"
        )
```


7.4 Migration Registry Pattern

Migrations are registered in a global migration registry:

Migration Registry

```
class Migration:
    def __init__(self, from_version: int, to_version: int,
                  automatic: bool, description: str):
        self.from_version = from_version
        self.to_version = to_version
        self.automatic = automatic
        self.description = description

    def migrate(self, index_path: Path) -> None:
        raise NotImplementedError

class MigrationRegistry:
    _migrations: list[Migration] = []

    @classmethod
    def register(cls, migration: Migration) -> None:
        cls._migrations.append(migration)

    @classmethod
    def get_migration(cls, from_v: int, to_v: int) -> Migration | None:
        for m in cls._migrations:
            if m.from_version == from_v and m.to_version == to_v:
                return m
        return None

# Register migrations
MigrationRegistry.register(Migration(
    from_version=1, to_version=2, automatic=True,
    description="Add chunk_overlap metadata field"
))
```

7.5 Breaking Change Policy

Version Policy

- **Minor versions** (e.g., 1 → 2): Auto-migrate on load. Changes are backward-compatible additions (new optional fields, metadata additions).
- **Major versions** (e.g., 1 → 10): Require manual migration via CLI command. Changes involve incompatible modifications (binary format changes, structural alterations).
- **Version increments**: Minor for additive changes; major for breaking changes.
- **Format changelog**: Maintained in Appendix A with detailed change descriptions.
- **Backup before migration**: The system automatically creates a backup of the index before any migration.

8 Async Operations and Long-Running Tasks

8.1 Overview

Operations that may take longer than a typical HTTP request timeout (index creation, bulk deletion, export, import, compaction) are executed asynchronously. The client receives an immediate response with a job identifier and can poll for status or subscribe to real-time progress updates.

8.2 Job Lifecycle

1. Client submits a long-running operation (e.g., POST `/api/v1/indexes`)
2. Server validates the request and creates a job record in SQLite
3. Server responds with **202 Accepted** and a `job_id` in the response body
4. The job is queued for execution by the background job scheduler
5. Client polls GET `/api/v1/jobs/{job_id}` for status updates
6. Alternatively, client subscribes to SSE for real-time progress
7. On completion, the job result is stored and available for retrieval
8. Job records are cleaned up after a configurable retention period (default: 7 days)

8.3 Job Status Endpoint

[GET `/api/v1/jobs/{job_id}`] **Response (Running):**

```
{
  "job_id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "index_create",
  "status": "running",
  "progress": {
    "percentage": 67,
    "message": "Embedding documents... (1015/1500)",
    "files_processed": 1015,
    "files_total": 1500
  },
  "created_at": "2025-01-15T10:30:00Z",
  "started_at": "2025-01-15T10:30:01Z",
  "estimated_completion": "2025-01-15T10:35:00Z"
}
```

Response (Completed):

```
{
  "job_id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "index_create",
  "status": "completed",
  "result": {
    "index_name": "my-docs",
    "vector_count": 15234,
    "file_count": 1500,
    "duration_seconds": 287.5,
    "index_size_mb": 12.3
  },
  "created_at": "2025-01-15T10:30:00Z",
  "started_at": "2025-01-15T10:30:01Z",
  "completed_at": "2025-01-15T10:34:48Z"
}
```

Response (Failed):

```
{
  "job_id": "550e8400-e29b-41d4-a716-446655440000",
  "type": "index_create",
  "status": "failed",
  "error": {
    "code": 500,
    "message": "Embedding failed for file 'corrupt.pdf'",
    "detail": "PyMuPDF error: Cannot open encrypted PDF",
    "file": "corrupt.pdf",
    "step": "embedding"
  },
  "created_at": "2025-01-15T10:30:00Z",
  "started_at": "2025-01-15T10:30:01Z",
  "failed_at": "2025-01-15T10:31:15Z"
}
```

```
}

```

8.4 SSE Progress Endpoint

For real-time progress updates, clients can subscribe to a Server-Sent Events (SSE) stream:

Protocol stream
`GET /api/v1/jobs/{job_id}/stream`

- Client opens an EventSource connection to the SSE endpoint
- Server sends events as they occur:

SSE Event Stream

```
event: progress
data: {"percentage": 25, "message": "Parsing files... (375/1500)"}

event: progress
data: {"percentage": 50, "message": "Embedding documents... (750/1500)"}

event: progress
data: {"percentage": 75, "message": "Building HNSW index..."}

event: completed
data: {"vector_count": 15234, "duration_seconds": 287.5}

event: error
data: {"code": 500, "message": "Embedding failed"}
```

Connection Management:

- Connection stays open until job completes or client disconnects
- Automatic reconnection: client retries with `Last-Event-ID` header
- Server sends keep-alive comments every 15 seconds to prevent proxy timeouts

8.5 Job Types

Job Type	Trigger	Result Fields
<code>index_create</code>	POST <code>/api/v1/indexes</code>	<code>index_name</code> , <code>vector_count</code> , <code>file_count</code> , <code>duration_seconds</code> , <code>index_size_mb</code>
<code>index_update</code>	POST <code>/api/v1/indexes/{name}/documents</code>	<code>vectors_added</code> , <code>duration_seconds</code>
<code>index_delete</code>	DELETE <code>/api/v1/indexes/{name}</code>	<code>vectors_removed</code> , <code>disk_freed_mb</code> , <code>duration_seconds</code>
<code>index_compact</code>	Auto-triggered or manual	<code>vectors_before</code> , <code>vectors_after</code> , <code>duration_seconds</code> , <code>memory_saved_mb</code>
<code>export</code>	POST <code>/api/v1/indexes/{name}/export</code>	<code>archive_path</code> , <code>archive_size_mb</code> , <code>duration_seconds</code>
<code>import</code>	POST <code>/api/v1/indexes/import</code>	<code>index_name</code> , <code>vector_count</code> , <code>duration_seconds</code>

8.6 Job Cleanup

- Completed and failed job records are retained for a configurable period (default: 7 days)
- Cleanup runs as a periodic background task (every 24 hours)
- Admin can configure retention via `jobs.retention_days`
- Export archive files are NOT auto-deleted (managed separately by the user)
- Job cleanup is idempotent and safe to run manually via CLI: `isocortex jobs cleanup`

9 Non-Functional Requirements

9.1 Performance Requirements

NFR-02: Indexing Throughput

- **Embedding throughput:** ≥ 500 documents/minute on a 4-core CPU
- **HNSW construction:** ≤ 5 minutes for 100K vectors
- Includes file parsing, chunking, embedding, and graph construction

NFR-03: API Response Time

- Non-search API endpoints: $< 200\text{ms}$ p95
- Health check endpoint: $< 50\text{ms}$
- Static asset serving (Web UI): $< 100\text{ms}$

9.2 Scalability Requirements

NFR-04: Index Size

- Support up to 10M vectors per index (memory limited)
- Multi-index: no hard limit on number of indices (limited by disk)

NFR-05: Concurrent Request Handling

- Support 10+ concurrent API requests without performance degradation
- Search throughput: ≥ 100 queries/second under concurrent load
- No deadlocks or race conditions under high concurrency
- Graceful degradation under extreme load (queue-based backpressure)

9.3 Memory Requirements

NFR-06: Memory Efficiency

IsoCortex must operate within configurable memory limits. This section defines memory requirements and the `max_memory_mb` parameter.

Vector Memory Table (float32, 384 dimensions):

Vectors	Raw Vectors	HNSW Overhead (3x)	Total Estimate	Min System RAM
10K	15 MB	45 MB	60 MB	2 GB
100K	147 MB	441 MB	588 MB	4 GB
1M	1.47 GB	4.41 GB	5.88 GB	8 GB
10M	14.7 GB	44.1 GB	58.8 GB	64 GB

Notes:

- Raw vector size = count × dimensions × 4 bytes (float32)
- HNSW graph overhead is approximately 2–4x the vector size (depends on *M* parameter)
- Total includes metadata, embedding cache, and Python runtime overhead (~200 MB)
- **Pre-indexing memory check:** Before index creation, the system estimates required memory and warns if it exceeds `max_memory_mb`
- If estimated memory exceeds 80% of available system RAM, a WARNING is logged and returned to the client
- If estimated memory exceeds `max_memory_mb`, the operation is rejected with a clear error

Configuration:

- `max_memory_mb`: Hard limit on total memory usage (default: 80% of system RAM)
- `memory.warning_threshold`: Fraction of `max_memory_mb` at which warnings trigger (default: 0.8)
- `embedding.cache_size`: Number of cached embeddings (default: 1000)

Minimum System Requirements

Resource	Minimum	Recommended
CPU	2 cores (x86_64)	4+ cores (x86_64, AVX2 support)
RAM	4 GB	8+ GB
Disk (installation)	1 GB	2 GB
Disk (per 100K vectors)	1 GB	2 GB (with headroom)
OS	Linux, macOS, Windows 10+	Ubuntu 22.04 LTS or equivalent
Python	3.10+	3.11+
Docker (optional)	20.10+	24.0+

9.4 Security Requirements

NFR-07: Authentication

- All API endpoints require JWT authentication (except login and first-run setup)
- Token expiry: 24 hours (access), 7 days (refresh)
- Tokens are cryptographically signed (HS256) with a server-secret key

NFR-08: Password Security

- bcrypt hashing with cost factor 12
- Minimum 12 characters with complexity requirements
- Account lockout after 5 failed attempts (15-minute window)
- Passwords never logged or exposed in error messages

NFR-09: Rate Limiting

- SQLite-backed sliding window rate limiting
- Default: 100 requests per minute per API key
- Search-specific: 60 searches per minute
- Admin-adjustable per key and per endpoint
- Persists across server restarts
- Works correctly across multiple Uvicorn workers

NFR-10: Input Validation

- All user inputs validated via Pydantic models (type checking + constraints)
- SQL injection prevented via parameterized queries
- Path traversal prevented via canonical path validation
- File upload size limited and type-validated

9.5 Reliability Requirements

NFR-11: Data Integrity

- Atomic index creation: either all vectors are indexed or none
- SQLite ACID compliance for all metadata operations
- SHA-256 checksums on exported archives for integrity verification
- Automatic index validation on load (check vectors.bin size against metadata)

NFR-12: Graceful Shutdown

- SIGTERM/SIGINT triggers graceful shutdown
- In-progress jobs are checkpointed and resumable
- SQLite WAL checkpoint on shutdown
- No data loss on crash during normal operation

NFR-13: Graceful Degradation

- **OOM handling:** If system runs out of memory during indexing, the operation is paused, an error is logged with a request_id, and a clear message is returned to the client
- **Partial results:** If batch search encounters a timeout on some queries, completed results are returned with error details for failed queries (207 Multi-Status)
- **Fallback behavior:** If the embedding cache cannot allocate memory, it is cleared and processing continues without caching
- **Circuit breaker:** If repeated failures occur on a specific index, it is temporarily marked unhealthy and searches are rejected with 503 until an admin intervenes

9.6 Usability Requirements

NFR-14: CLI Usability

- All commands include `-help` with examples
- Human-readable output by default; `-json` flag for machine output
- Color-coded output (auto-disabled in non-TTY environments)
- Progress bars for long-running operations

NFR-15: API Documentation

- Auto-generated OpenAPI 3.0 documentation via FastAPI
- Interactive Swagger UI at `/docs`
- ReDoc at `/redoc`
- All endpoints documented with request/response examples

NFR-16: Web UI Responsiveness

- Responsive design supporting desktop, tablet, and mobile viewports
- Page load time: < 2 seconds on broadband
- Search results displayed within 500ms of query submission

9.7 Embedding Cache Requirements

NFR-17: Embedding Cache Hit Rate

- Target cache hit rate: > 80% for repeated queries within a session
- Cache hit rate is logged periodically (every 1000 queries)
- Cache metrics exposed via the analytics endpoint: hits, misses, hit_rate, eviction_count
- Cache performance should not degrade overall search latency (cache lookup < 1ms)

9.8 Portability Requirements

NFR-18: Cross-Platform

- Runs on Linux (Ubuntu 20.04+), macOS (12+), Windows 10+
- Docker images available for all major platforms (linux/amd64, linux/arm64)
- No platform-specific code in core modules
- File paths handled via `pathlib.Path` for cross-platform compatibility

10 Error Handling Specification

10.1 Standard Error Format

All API errors follow a consistent envelope format:

Standard Error Response

```
{
  "error": "Index not found",
}
```

```
{
  "code": 404,
  "detail": "No index named 'nonexistent' exists in this instance.",
  "request_id": "550e8400-e29b-41d4-a716-446655440000"
}
```

Fields:

- **error** (string): Human-readable error summary
- **code** (integer): HTTP status code for programmatic handling
- **detail** (string): Detailed explanation with context-specific information
- **request_id** (string): Unique identifier for debugging and log correlation

10.2 Request ID Generation

- Every incoming request receives a unique UUID v4 as its **request_id**
- Generated in middleware before route handling
- Included in response headers (**X-Request-ID**)
- Included in all log entries for that request
- Propagated to background jobs spawned by the request
- Enables end-to-end request tracing across async boundaries

10.3 Error Codes Reference

Code	Status	Error	When Used
400	Bad Request	BAD_REQUEST	Invalid request body, missing required fields, validation errors
401	Unauthorized	UNAUTHORIZED	Missing or invalid JWT token, expired token
403	Forbidden	FORBIDDEN	Insufficient permissions (non-admin accessing admin endpoints), locked account
404	Not Found	NOT_FOUND	Index, document, user, or job not found
409	Conflict	CONFLICT	Duplicate index name, setup already completed, version conflict
422	Unprocessable Entity	VALIDATION_ERROR	Pydantic validation failure (detailed field errors)
429	Rate Limited	RATE_LIMITED	Request rate exceeded; includes retry_after field
500	Internal Error	INTERNAL_ERROR	Unexpected server error (logged with full traceback)
503	Unavailable	SERVICE_UNAVAILABLE	Server overloaded, embedder queue full, index marked unhealthy

10.4 Structured Error Logging

Structured Log Entry (JSON output mode)

```
{
  "timestamp": "2025-01-15T10:30:15.123Z",
  "level": "ERROR",
  "request_id": "550e8400-e29b-41d4-a716-446655440000",
}
```



```
{
  "method": "POST",
  "path": "/api/v1/indexes/my-docs/search",
  "status": 500,
  "error": "INTERNAL_ERROR",
  "message": "HNSW search failed: memory allocation error",
  "duration_ms": 45,
  "user_id": "user-uuid",
  "index_name": "my-docs",
  "stack_trace": "Traceback (most recent call last)..."
}
```

- All errors logged with structured JSON format (in production)
- `request_id` enables correlation between HTTP response and server logs
- Stack traces included only at ERROR level (not exposed to clients)
- Log levels: DEBUG, INFO, WARNING, ERROR, CRITICAL
- Configurable via `LOG_LEVEL` environment variable

11 Rate Limiting

11.1 SQLite-Backed Sliding Window

IsoCortex implements rate limiting using **sliding window counters stored in SQLite**, providing persistence across restarts and correct behavior across multiple Uvicorn workers.

Rate Limiting Design

Mechanism:

- Each request increments a counter in a SQLite table keyed by (API key, endpoint, time window)
- Sliding window: counts are aggregated over the last 60 seconds
- Query: `SELECT COUNT(*) FROM rate_limits WHERE key = ? AND endpoint = ? AND timestamp > datetime('now', '-60 seconds')`
- SQLite WAL mode ensures non-blocking concurrent access

Advantages over in-memory:

- ✓ Persists across server restarts (no reset on deployment)
- ✓ Shared across multiple Uvicorn workers (single source of truth)
- ✓ No memory overhead for counters
- ✓ Auditable: admins can inspect rate limit history

Response Headers:

- `X-RateLimit-Limit`: Maximum requests per window
- `X-RateLimit-Remaining`: Remaining requests in current window
- `X-RateLimit-Reset`: Unix timestamp when the window resets
- `Retry-After`: Seconds until the client can retry (on 429 responses)

11.2 Admin Rate Limit Management

[GET /api/v1/admin/rate-limits] **Description:** View current rate limit status for all API keys and endpoints.

Response:

```
{
  "limits": [
```

```
{
  {
    "key": "user-uuid-1",
    "endpoint": "*",
    "limit": 100,
    "remaining": 67,
    "reset_at": "2025-01-15T10:31:00Z"
  },
  {
    "key": "user-uuid-1",
    "endpoint": "/api/v1/indexes/*/search",
    "limit": 60,
    "remaining": 45,
    "reset_at": "2025-01-15T10:31:00Z"
  }
}
```

11.3 Configuration

Parameter	Default	Scope	Description
rate_limit.default	100/min	Global	Default requests per minute
rate_limit.search	60/min	Search endpoints	Search-specific limit
rate_limit.write	30/min	Write endpoints	Index/document mutation limit
rate_limit.burst	20	Global	Burst allowance (first N requests instant)

12 Known Limitations

The following limitations are acknowledged for IsoCortex v1.0 and may be addressed in future releases:

13 Development Phases

Development is organized into logical phases representing progressive capability delivery. Each phase builds upon the previous and delivers a usable subset of functionality.

Phase Overview

Phase	Focus Area	Key Deliverables
Phase 1	Core Engine	- File extraction pipeline (7 formats) - Text chunking with overlap - MiniLM embedding provider with download + cache - HNSW index construction and search - EmbeddingProvider ABC and registry - FileExtractor ABC and registry - Index serialization to disk (vectors.bin, meta-data.json, hnsw_index.bin) - Index format versioning system - Soft-delete with compaction - ReadWriteLock for HNSW - Embedding queue + LRU cache
Phase 2	Interface Layer	- FastAPI application setup - REST API: index CRUD, search, documents, batch search - Pagination (offset + cursor-based) - Standard error format with request_id - CLI (Typer): index, search, serve commands - OpenAPI documentation
Phase 3	Platform Features	- Web UI (Next.js SPA) - Multi-index management - Incremental index updates - Export/Import (.isocortex archives) - Async job system (202 Accepted + polling + SSE) - Analytics engine (search logging, usage metrics)
Phase 4	Enterprise Features	- JWT authentication - User management (CRUD, roles, logout) - First-run setup wizard - SQLite-backed rate limiting - Admin API endpoints - Per-key rate limit configuration
Phase 5	Deployment & Polish	- Docker multi-stage build (API + Web images) - Docker Compose configuration - CI/CD pipeline (GitHub Actions) - Memory requirements system (pre-check, warnings) - Comprehensive test suite - Documentation (user guide, API reference) - Performance benchmarking - Error handling hardening

14 Deployment

14.1 Docker Images

IsoCortex provides two optimized Docker images built with multi-stage builds:

Docker Image Strategy

Image Split:

- `isocortex-api` : Full API server with embedding model (< 2.5 GB)
- `isocortex-web` : Frontend-only SPA (< 500 MB)

Optimizations:

- PyTorch CPU-only: `pip install torch --index-url https://download.pytorch.org/whl/cpu`
- Multi-stage build: separate C++ builder stage (compiles native extensions) and Python runtime stage
- `.dockerignore` excludes: `.git`, `tests`, `docs`, `*.pyc`, `~`, `node_modules`, `.env`
- Layer caching: model download in its own layer (changes only when model version changes)
- Base image: `python:3.11-slim` for minimal footprint

Multi-Stage Dockerfile:

Dockerfile (Simplified)

```
# Stage 1: Builder
FROM python:3.11-slim AS builder
RUN apt-get update && apt-get install -y --no-install-recommends \
    build-essential gcc g++
COPY pyproject.toml .
RUN pip install --no-cache-dir torch --index-url \
    https://download.pytorch.org/whl/cpu
RUN pip install --no-cache-dir .

# Stage 2: Runtime
FROM python:3.11-slim AS runtime
RUN apt-get update && apt-get install -y --no-install-recommends \
    libgomp1 tesseract-ocr && rm -rf /var/lib/apt/lists/*
COPY --from=builder /usr/local/lib/python3.11/site-packages /usr/local/lib/python3.11/
site-packages
COPY --from=builder /usr/local/bin /usr/local/bin
COPY src/ /app/src/
COPY cli/ /app/cli/
WORKDIR /app
EXPOSE 8000
CMD ["uvicorn", "isocortex.api:app", "--host", "0.0.0.0", "--port", "8000"]
```

14.2 Docker Compose

docker-compose.yml

```
version: "3.9"
services:
  api:
    image: isocortex-api:latest
    ports:
      - "8000:8000"
    volumes:
      - isocortex-data:/app/data
      - isocortex-models:/root/.isocortex/models
    environment:
      - ISOCORTEX_MAX_MEMORY_MB=4096
      - ISOCORTEX_LOG_LEVEL=INFO
      - ISOCORTEX_SECRET_KEY=${SECRET_KEY}
```

```

restart: unless-stopped

web:
  image: isocortex-web:latest
  ports:
    - "3000:80"
  depends_on:
    - api
  environment:
    - NEXT_PUBLIC_API_URL=http://api:8000
  restart: unless-stopped

volumes:
  isocortex-data:
  isocortex-models:

```

14.3 Environment Variables

Variable	Default	Required	Description
ISOCORTEX_HOST	0.0.0.0	No	API bind address
ISOCORTEX_PORT	8000	No	API port
ISOCORTEX_WORKERS	1	No	Uvicorn worker count
ISOCORTEX_SECRET_KEY	—	Yes	JWT signing key (auto-generated on first run if not set)
ISOCORTEX_MAX_MEMORY_MB	Auto	No	Maximum memory limit in MB
ISOCORTEX_LOG_LEVEL	INFO	No	Logging level
ISOCORTEX_DATA_DIR	./data	No	Data directory path
ISOCORTEX_MODEL_DIR	~/.isocortex/models	No	Model cache directory

15 Monetization and Commercial Deployment

15.1 Pricing Tiers

IsoCortex follows a feature-gated, self-hosted licensing model. Customers deploy IsoCortex on their own infrastructure; pricing is based on feature access, support level, and usage capacity rather than per-query billing. This eliminates data sovereignty concerns while enabling predictable costs.

Pricing Tier Specification				
Feature	Community	Professional		Enterprise
Price	Free	\$49/month \$490/year	or	Custom
Max Vectors/Index	100K	1M		10M+
Max Indexes	3	Unlimited		Unlimited
Users	2	10		Unlimited
Embedding Models	MiniLM only	MiniLM + custom		Multi-model runtime
Search API	✓	✓		✓
Batch Search	✓ (max 10)	✓ (max 50)		✓ (max 200)
Rate Limits	30/min	100/min		Custom
Analytics	Basic	Full		Full + audit log
Export/Import	✓	✓		✓
Custom Extractors	✗	✓		✓
SSO/SAML	✗	✗		✓
API Keys (per user)	1	5		Unlimited
GPU Acceleration	✗	✗		✓
SLA Support	Community forum	Email (24h response)		Dedicated (4h response)
Multi-Tenancy	✗	✗		✓
FIPS Compliance	✗	✗		✓

Tier Enforcement Mechanism:

- Tier is determined by a signed license key stored in `~/.isocortex/license.key`
- License key contains: tier identifier, vector cap, user cap, feature flags, expiry date, and an HMAC-SHA256 signature
- The API server validates the license on startup and enforces limits at the service layer
- Exceeded limits return 429 `PAYLOAD_TOO_LARGE` (vectors) or 403 `FEATURE_LOCKED` (features)
- Community tier is activated by default when no license key is present
- License keys can be rotated via the admin API without downtime

License Key Format:

```
{
  "tier": "professional",
  "max_vectors_per_index": 1000000,
  "max_indexes": 0,
  "max_users": 10,
  "max_batch_queries": 50,
  "features": ["custom_extractors", "full_analytics"],
  "issued_at": "2025-01-15T00:00:00Z",
  "expires_at": "2026-01-15T00:00:00Z",
  "customer_id": "cust_abc123",
  "signature": "hmac-sha256-hex..."
}
```

```
}

```

15.2 Multi-Tenancy Architecture

Enterprise deployments require tenant isolation to serve multiple organizations or departments from a single IsoCortex instance. The multi-tenancy model provides logical isolation without the overhead of separate processes or containers.

Multi-Tenancy Design

Tenant Isolation Model:

- **Tenant ID:** Each tenant is identified by a UUID (e.g., `tenant_1234abcd`) stored in the JWT claims
- **Data isolation:** Each tenant's indices are stored in separate directories under `~/.isocortex/tenants/{tenant_id}/`
- **User scoping:** Users belong to exactly one tenant; all API operations are scoped to the caller's tenant
- **No cross-tenant access:** A user from Tenant A cannot query, list, or modify indices belonging to Tenant B
- **Shared infrastructure:** Embedding model cache, job scheduler, and rate limiter are shared across tenants (resource pool)

Tenant Management (Enterprise Admin Only):

- `POST /api/v1/admin/tenants` — Create a new tenant (name, description, tier override)
- `GET /api/v1/admin/tenants` — List all tenants with usage summaries
- `GET /api/v1/admin/tenants/{tenant_id}` — Get tenant details and resource usage
- `PUT /api/v1/admin/tenants/{tenant_id}` — Update tenant configuration
- `DELETE /api/v1/admin/tenants/{tenant_id}` — Archive and soft-delete a tenant

Per-Tenant Resource Limits:

Resource	Default per Tenant	Configurable
Max vectors (across all indexes)	1M	Yes
Max indexes	50	Yes
Max users	25	Yes
Max storage (disk)	10 GB	Yes
Search rate limit	100/min	Yes
Embedding queue priority	Normal	Yes

JWT Claims Extension for Multi-Tenancy:

```
{
  "sub": "user-uuid",
  "role": "user",
  "tenant_id": "tenant_1234abcd",
  "tenant_name": "Acme Corp",
  "tier": "enterprise",
  "exp": 1735689600
}
```

15.3 Billing Infrastructure

IsoCortex uses a usage-metered, subscription-based billing model. Billing data is collected locally and reported to the licensing server for invoicing. No customer data leaves the customer's

infrastructure — only aggregate usage counters are transmitted.

Billing Architecture

Usage Meters (collected locally):

- **Total vectors indexed:** Cumulative count of vectors across all tenant indexes
- **Search queries executed:** Count of all search API calls (single + batch, counted per query)
- **Active users:** Number of unique users who made at least one API call in the billing period
- **Peak concurrent users:** Maximum simultaneous API connections observed
- **Storage consumed:** Total disk usage for indices, metadata, and analytics (in GB)
- **Uptime percentage:** Fraction of time the API server was reachable (5-minute intervals)

Billing Cycle:

- Monthly billing cycle aligned with the license activation date
- Usage report generated automatically on the last day of each cycle
- Report uploaded to the licensing server via `POST /api/v1/usage/report` (authenticated)
- Offline mode: If the server cannot reach the licensing server, usage is cached locally and retried with exponential backoff (max 7 days)
- Grace period: 14 days after invoice due date before tier is downgraded to Community

Usage Report Format:

```
{
  "customer_id": "cust_abc123",
  "license_key": "lic_...",
  "billing_period": {
    "from": "2025-01-01T00:00:00Z",
    "to": "2025-01-31T23:59:59Z"
  },
  "usage": {
    "total_vectors_indexed": 452000,
    "search_queries_executed": 128450,
    "active_users": 8,
    "peak_concurrent_users": 3,
    "storage_consumed_gb": 4.7,
    "uptime_percentage": 99.97
  },
  "signature": "hmac-sha256-hex..."
}
```

Admin Usage Dashboard:

- `GET /api/v1/admin/usage/current` — Current billing period usage with projections
- `GET /api/v1/admin/usage/history?months=6` — Historical usage for trend analysis
- `GET /api/v1/admin/usage/export?format=csv` — Export usage data for external tools
- `POST /api/v1/admin/usage/report` — Manually trigger usage report upload

15.4 Compliance and Regulatory Requirements

Enterprise customers in regulated industries (healthcare, finance, government) require documented compliance with security and privacy standards. IsoCortex's self-hosted architecture provides a strong foundation, and the following compliance measures are specified.

Compliance Specification

SOC 2 Type II Readiness:

- **Access control:** Role-based access with audit logging (all admin actions logged with user ID, timestamp, and action details)
- **Data encryption:** TLS 1.3 for all API communication (configurable via `ssl.cert` and `ssl.key`); at-rest encryption of index files via OS-level full-disk encryption (documented as customer responsibility)
- **Change management:** All configuration changes logged; schema migrations versioned and reversible
- **Incident response:** Error codes and request IDs enable log-based incident investigation; structured JSON logging for SIEM integration
- **Data retention:** Configurable retention policies for analytics (default: 90 days), job records (default: 7 days), and audit logs (default: 1 year)

GDPR Considerations:

- **Data minimization:** Query text is SHA-256 hashed before analytics storage (irreversible); no IP addresses logged; user IDs are UUIDs (no PII)
- **Right to erasure:** `DELETE /api/v1/auth/users/{id}` cascades to remove all user data including analytics records, audit logs, and personal configuration
- **Data portability:** Full index export (`POST /api/v1/indexes/{name}/export`) enables data portability in open format
- **Self-hosted advantage:** Customer retains full control over data residency; no data is transmitted to external services except aggregate, anonymized usage counters for billing

HIPAA Considerations (Enterprise Tier):

- **BAA requirement:** Business Associate Agreement offered as part of Enterprise tier onboarding
- **Audit logging:** Comprehensive audit trail of all data access events (who accessed which index, when, and how many results)
- **Access controls:** Integration with external identity providers (SAML 2.0 / OIDC) for centralized user management
- **Encryption:** AES-256 at-rest encryption for index files via optional transparent encryption layer (FIPS 140-2 validated module in Enterprise tier)
- **Disposal:** Index deletion securely overwrites vector and metadata files before removal (3-pass overwrite)

FIPS 140-2 Compliance (Enterprise Tier):

- **Cryptographic module:** Uses FIPS-validated OpenSSL for TLS, JWT signing (HMAC-SHA256), and password hashing (bcrypt)
- **Configuration:** Enabled via `security.fips_mode = true` in configuration; enforces FIPS-approved algorithms only
- **Restrictions:** In FIPS mode, non-FIPS algorithms (e.g., MD5, SHA-1) are disabled even if referenced in legacy index formats

Audit Log API (Enterprise Tier):

- `GET /api/v1/admin/audit-logs?from=...to=...user=...` — Query audit trail with filtering
- `GET /api/v1/admin/audit-logs/export?format=siem` — Export in SIEM-compatible format (CEF/JSON)
- Audit events include: login/logout, user CRUD, index CRUD, role changes, configuration

changes, data exports, and failed access attempts

- Audit logs stored in separate SQLite database with write-once semantics (no deletion, only retention-based cleanup)

A Index Format Specification

A.1 Directory Layout

Index Directory Structure

```
my-docs/
  index_info.json      # Index metadata and format version
  vectors.bin          # Binary vector data
  metadata.json        # Document chunk metadata
  hnsw_index.bin       # HNSW graph structure
```

A.2 index_info.json Schema

```
{
  "format_version": 1,
  "name": "my-docs",
  "description": "Company documentation index",
  "created_at": "2025-01-15T10:30:00Z",
  "updated_at": "2025-01-15T12:00:00Z",
  "embedding_model": "all-MiniLM-L6-v2",
  "embedding_dimension": 384,
  "vector_count": 15234,
  "deleted_count": 47,
  "hnsw_params": {
    "M": 16,
    "ef_construction": 200,
    "ef_search": 50,
    "metric": "cosine"
  },
  "chunk_config": {
    "chunk_size": 512,
    "chunk_overlap": 50
  },
  "file_hashes": {
    "sha256_of_source_files": "abc123..."
  }
}
```

A.3 vectors.bin Binary Layout

Offset	Size	Description
0	4 bytes	Magic number: 0x49534F56 (“ISoV” in ASCII)
4	4 bytes	Format version (uint32, little-endian)
8	4 bytes	Vector count N (uint32, little-endian)
12	4 bytes	Dimension D (uint32, little-endian)
16	4 bytes	Element type: 0=float32, 1=float16 (uint32)
20	$N \times D \times 4$ bytes	Raw vector data (float32, row-major)

Binary Notes

- All integers are little-endian (standard x86/ARM)
- Vectors are stored sequentially: vector 0 occupies bytes $[20, 20 + D \times 4)$, vector 1 occupies bytes $[20 + D \times 4, 20 + 2 \times D \times 4)$, etc.
- Deleted vectors are NOT removed from vectors.bin; their status is tracked in metadata.json
- Total file size: $20 + N \times D \times 4$ bytes

- For 100K vectors at 384 dimensions: $20 + 100,000 \times 384 \times 4 = 153,600,020$ bytes (≈ 146.5 MB)

A.4 metadata.json Schema

```
{
  "chunks": [
    {
      "id": "550e8400-e29b-41d4-a716-446655440000",
      "vector_index": 0,
      "deleted": false,
      "source_file": "docs/auth.md",
      "file_extension": ".md",
      "page": null,
      "section": "## Authentication Flow",
      "chunk_index": 0,
      "text_preview": "Authentication is handled by JWT tokens...",
      "token_count": 487,
      "created_at": "2025-01-15T10:30:05Z"
    }
  ],
  "total_chunks": 15234,
  "total_deleted": 47
}
```

A.5 hnsw_index.bin Format

Offset	Size	Description
0	4 bytes	Magic number: 0x484E5357 (“HNSW”)
4	4 bytes	Format version (uint32)
8	4 bytes	Element count N (uint32)
12	4 bytes	M parameter (uint32)
16	4 bytes	M_{max0} (uint32, $M \times 2$ for layer 0)
20	4 bytes	$ef_construction$ (uint32)
24	4 bytes	M_max (uint32, multilayer)
28	4 bytes	Max layer (uint32)
32	1 byte	Metric type: 0=cosine, 1=l2, 2=inner_product
33	$N \times 4$ bytes	Entry point assignments (uint32 per element)
$33 + N \times 4$	$N \times 4$ bytes	Layer assignments (uint32 per element)
$33 + N \times 8$	variable	Graph adjacency lists (compressed format)

A.6 Format Version Changelog

Version	Change	Migration
1	Initial format	N/A (baseline)
2 (future)	Add chunk_overlap to metadata	Auto-migrate (additive field)
3 (future)	Switch vectors.bin to float16	Manual (requires re-encode)

B Configuration Reference

All configuration can be provided via:

1. Environment variables (prefixed with `ISOCORTEX_`)
2. Configuration file (`isocortex.yaml` or `isocortex.json`)
3. CLI flags (highest priority)

Priority order: CLI flags > Environment variables > Configuration file > Defaults.

Parameter	Type	Default	Constraints	Description
<i>Server</i>				
host	string	0.0.0.0	Valid IP/host-name	API bind address
port	int	8000	1–65535	API port
workers	int	1	1–16	Uvicorn worker count
log_level	string	INFO	DEBUG/INFO/WARN/ERROR	Logging verbosity
secret_key	string	auto-gen	Min 32 chars	JWT signing key
<i>Data</i>				
data_dir	string	./data	Writable path	Data storage directory
model_dir	string	~/.isocortex/models	Writable path	Model cache directory
max_memory_mb	int	auto	512–max_ram	Maximum memory limit
memory.warning_threshold	float	0.8	0.1–1.0	Fraction for memory warnings
<i>Embedding</i>				
embedding.model	string	all-MiniLM-L6-v2	Valid model ID	Default embedding model
embedding.cache_size	int	1000	0–100000	LRU cache entry limit
embedding.queue_size	int	500	10–10000	Embedder request queue depth
embedding.batch_size	int	32	1–256	Batch inference size
<i>HNSW</i>				
hnsw.M	int	16	4–128	Graph connectivity
hnsw.ef_construction	int	200	50–2000	Construction candidate list
hnsw.ef_search	int	50	10–500	Search candidate list
hnsw.metric	string	cosine	cosine/l2/ip	Distance metric
hnsw.compaction.threshold	float	0.10	0.01–0.50	Tombstone ratio for compaction
<i>Chunking</i>				
chunk.size	int	512	64–4096	Tokens per chunk
chunk.overlap	int	50	0–500	Token overlap between chunks

Parameter	Type	Default	Constraints	Description
chunk.min_size	int	10	1–100	Minimum tokens per chunk
<i>Rate Limiting</i>				
rate_limit.default	int	100	1–10000	Requests per minute (global)
rate_limit.search	int	60	1–10000	Requests per minute (search)
rate_limit.write	int	30	1–10000	Requests per minute (write)
rate_limit.burst	int	20	1–1000	Burst allowance
<i>Jobs</i>				
jobs.retention_days	int	7	1–365	Job record retention period
jobs.max_concurrent	int	2	1–8	Max concurrent jobs
jobs.cleanup_interval_hrs	int	24	1–168	Cleanup task interval
<i>Search</i>				
search.default_k	int	5	1–100	Default results per query
search.max_k	int	100	1–1000	Maximum k allowed
search.page_size_default	int	10	1–100	Default pagination size
search.page_size_max	int	100	1–100	Maximum pagination size (server-enforced)
search.batch.max_queries	int	50	1–200	Max queries per batch
search.batch.timeout	int	30	1–300	Per-query timeout (seconds)
search.batch.concurrency	int	4	1–16	Parallel query processing
<i>Security</i>				
auth.token_expiry_hours	int	24	1–72	Access token TTL
auth.refresh_expiry_days	int	7	1–30	Refresh token TTL
auth.bcrypt_cost	int	12	10–14	bcrypt cost factor
auth.lockout_attempts	int	5	1–20	Failed attempts before lockout
auth.lockout_minutes	int	15	1–60	Lockout duration
auth.password_min_length	int	12	8–128	Minimum password length

C API Error Codes

Complete reference for all error codes returned by the IsoCortex API.

	HTTP	Code	Category	Description
<i>Client Errors (4xx)</i>				
400		BAD_REQUEST	Validation	Malformed request syntax, invalid parameters
400		QUERY_TOO_SHORT	Search	Query text below minimum length (3 chars)
400		INVALID_FILTER	Search	Malformed filter expression
400		BATCH_TOO_LARGE	Batch	Exceeds max_queries per batch
401		MISSING_TOKEN	Auth	No Authorization header provided
401		INVALID_TOKEN	Auth	JWT token is malformed or tampered
401		EXPIRED_TOKEN	Auth	JWT token has expired
401		INVALID_CREDENTIALS	Auth	Username or password incorrect
403		FORBIDDEN	Auth	Insufficient role permissions
403		ACCOUNT_LOCKED	Auth	Account temporarily locked (too many failed attempts)
403		SETUP_COMPLETED	Auth	First-run setup already completed
404		INDEX_NOT_FOUND	Index	Specified index name does not exist
404		DOCUMENT_NOT_FOUND	Document	Specified document chunk ID does not exist
404		USER_NOT_FOUND	User	Specified user ID does not exist
404		JOB_NOT_FOUND	Job	Specified job ID does not exist
409		INDEX_EXISTS	Index	An index with this name already exists
409		USER_EXISTS	User	A user with this username already exists
409		VERSION_CONFLICT	Index	Index format version is incompatible
422		VALIDATION_ERROR	Validation	Pydantic schema validation failure
429		RATE_LIMITED	Rate Limit	Request rate exceeded; check Retry-After header
<i>Server Errors (5xx)</i>				
500		INTERNAL_ERROR	General	Unexpected server error (check request_id in logs)
500		EMBEDDING_FAILED	Embedding	Model inference error
500		INDEX_CORRUPTED	Index	Index data failed validation on load
500		MIGRATION_FAILED	Index	Format migration failed
500		EXTRACTION_FAILED	Ingestion	File extraction/parsing error
503		SERVICE_UNAVAILABLE	General	Server overloaded or embedder queue full
503		INDEX_UNHEALTHY	Index	Index marked unhealthy (circuit breaker)
503		MODEL_NOT_LOADED	Embedding	Embedding model not yet loaded/available

D Glossary

Term	Definition
ABC	Abstract Base Class — a Python class that cannot be instantiated and defines methods that subclasses must implement
Approximate Nearest Neighbor (ANN)	Search algorithms that return approximately correct nearest neighbors, trading some accuracy for significant speed gains
Backpressure	A flow-control mechanism where a system signals upstream producers to slow down when it cannot process requests fast enough
bcrypt	A password hashing function designed to be computationally expensive, resisting brute-force and rainbow table attacks
Chunk	A segment of text extracted from a document, typically 200–1000 tokens, optimized for embedding and search relevance
Compaction	The process of rebuilding an HNSW index to remove soft-deleted (tombstoned) vectors and reclaim memory
Cosine Similarity	A measure of similarity between two vectors calculated as the cosine of the angle between them; range $[-1, 1]$ where 1 means identical
Cursor-based Pagination	A pagination method using opaque tokens (cursors) instead of numeric offsets, providing consistent results even when data changes between requests
Embedding	A dense numerical vector representation of text that captures semantic meaning; used for similarity comparison
Embedding Provider	A pluggable module in IsoCortex that implements the EmbeddingProvider ABC to generate embeddings using a specific model
Extractor	A plugin module that implements the FileExtractor ABC to parse specific file formats into structured text chunks
HNSW	Hierarchical Navigable Small World — a multi-layered proximity graph structure enabling efficient approximate nearest neighbor search
JWT	JSON Web Token — a compact, URL-safe means of representing claims between two parties, used for authentication
LRU Cache	Least Recently Used cache — an eviction strategy that removes the least recently accessed items when the cache reaches capacity
MiniLM	A compact sentence transformer model (all-MiniLM-L6-v2) producing 384-dimensional embeddings with good quality-to-speed ratio
Offset-based Pagination	A pagination method using page numbers and sizes to navigate result sets; simpler but may skip/duplicate items on concurrent modifications
ReadWriteLock	A synchronization primitive allowing multiple concurrent readers OR a single exclusive writer, preventing data corruption during concurrent access
Registry Pattern	A design pattern where components register themselves with a central registry for dynamic lookup and instantiation
SSE (Server-Sent Events)	A protocol allowing servers to push real-time updates to clients over HTTP using a persistent connection

Term	Definition
SQLite WAL	Write-Ahead Logging mode for SQLite that enables concurrent readers and a single writer without blocking
Soft Delete	A deletion pattern where records are marked as deleted (tombstoned) rather than physically removed from storage
Tombstone	A marker indicating a vector or record has been soft-deleted and should be excluded from search results
Token	A unit of text produced by a tokenizer; roughly equivalent to a word or subword (depends on the tokenizer used)
Uvicorn	An ASGI web server implementation for Python, used to run FastAPI applications
WAL Checkpoint	The process of writing WAL journal entries back into the main SQLite database file to reclaim disk space
